KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

Personal Contact Directory with Search and Update Features

Submitted by

RAGUL R

25BIT042

Under the Guidance of

DHILEEBAN A

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026 Semester: **II**

DECLARATION

I hereby declare that the project entitled **"Personal Contact Directory with Search and Update Features"** submitted in partial fulfillment of the requirements for the course **JAVA PROGRAMMING** is a **record of my original work** carried out under the guidance of , Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

RAGUL R

(Your Name)

CERTIFICATE

This is to certify that the project entitled **"Personal Contact Directory with Search and Update Features "** is a **bona fide work** carried out by **[RAGUL R] [I-BSc IT-.'A']**, **[]**, **KG College of Arts & Science**, in partial fulfillment of the requirements for the course **[JAVA PROGRAMMING]**.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place: COIMBATORE

Date: 09.04.2026

Submitted for the Viva-Voce Examination held on **KG COLLAGE OF ARTS & SCIENCE**

Guide:
(Technical Trainer, KGCAS)

Head of the Department
(Technical Head, KGCAS)

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	
1.1	Objective	
1.2	Overview	
1.3	Features	
2	**Program Analysis**	
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scopes and Limitations	
3	**System Design**	
3.1	Modules	
4	**Methodology**	
4.1	Problem Analysis	
4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	
4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	**Source Code**	
6	**Input and Output Screens**	
7	**Discussion and Conclusion**	
7.1	Discussion	
7.2	Conclusion	
8	**References**	

ABSTRACT

The **Personal Contact Directory with Search and Update Features** is a Java-based application developed to efficiently manage and organize personal contact information. In many situations, contacts are stored manually or scattered across different platforms, leading to difficulties in searching, duplication of data, loss of important information, and inefficient management.

This project aims to simplify contact handling by providing an automated system that allows users to store, retrieve, update, and delete contact details easily. The system enables users to manage contacts through a menu-driven interface, ensuring quick access and better organization of data.

The application is developed using Core Java programming concepts such as classes, objects, methods, loops, conditional statements, and file handling. It can be implemented as a console-based application or further enhanced using a graphical user interface with Java Swing.

The main objective of the system is to improve efficiency, reduce manual effort, and ensure secure and structured storage of contact information. The system provides features such as adding new contacts, searching existing contacts, updating details, and deleting records, making it suitable for personal use, small offices, and educational purposes.

This project demonstrates the practical implementation of object-oriented programming in Java and provides a foundation for developing advanced contact management systems with features like database integration, cloud storage, and enhanced security in the future.

1.INTRODUCTION

In today's digital world, managing personal and professional contacts efficiently is very important. The **Personal Contact Directory with Search and Update Features** is a software application developed to store, organize, and manage contact information in a structured and user-friendly manner.

This project is designed as a simple application that allows users to perform operations such as adding new contacts, searching for existing contacts, updating contact details, and deleting unwanted records. It helps in maintaining all contact information in one place, avoiding the problems of manual record keeping such as data loss, duplication, and difficulty in searching.

The system can be implemented as a console-based application using programming languages like Java, Python, or C++. It uses basic programming concepts such as arrays or collections, loops, conditional statements, and functions to perform different operations.

The main objective of this project is to reduce manual effort and provide quick access to contact details. The search feature allows users to find contacts instantly, while the update feature ensures that information can be modified whenever needed.

This project is useful for students, individuals, and small offices to manage their contacts efficiently. It also demonstrates the practical application of programming concepts and serves as a foundation for developing advanced contact management systems with features like database integration, graphical user interface, and cloud storage.

1.1 Objective

The main objective of this project is to develop a Personal Contact Directory system that allows users to store, search, update, and manage contact information efficiently. The system aims to simplify contact management by providing quick access and modification features.

1.2 Overview

A Personal Contact Directory is a software application designed to maintain a collection of contact details such as names, phone numbers, email addresses, and addresses. This project focuses on creating a user-friendly interface with essential functionalities like adding, searching, updating, and deleting contacts.

1.3 Features

- Add new contacts
- Search contacts by name or number
- Update existing contact details

- Delete contacts
- Display all contacts

2. Program Analysis

2.1 Problem Definition

Managing contacts manually or in scattered formats can be inefficient and error-prone. There is a need for a centralized system that allows easy storage, retrieval, and modification of contact information.

2.2 Project Overview

The project provides a digital solution for storing personal contacts. It allows users to perform operations like insertion, searching, updating, and deletion using a structured approach.

2.3 Scope and Limitations

Scope:

- Can be used for personal or small-scale contact management
- Simple and easy to use
- Can be extended with GUI or database integration

Limitations:

- Limited storage if implemented using basic data structures
- No cloud synchronization (in basic version)
- Security features may be minimal

3.SYSTEM DESIGN

System design is the stage where the structure and components of the system are planned before the actual implementation. In this phase, the system is divided into different modules, and the workflow of the application is defined. Proper system design helps in organizing the program efficiently and makes development easier.

The **Personal Contact Directory with Search and Update Features** is designed as a menu-driven application where users can select food items, enter quantities, and generate a final bill automatically.

The system is divided into several modules to perform specific tasks. Each module is responsible for handling a particular function of the system.

System design defines the structure and working of the system

---

3.1 Modules

- **Add Contact Module – Adds new contact details**
- **Search Module – Searches contacts using name or number**
- **Update Module – Modifies existing contact details**
- **Delete Module – Removes contacts**
- **Display Module – Shows all saved contacts**

4.METHODOLOGY

Methodology describes the process followed to develop the project. It explains the steps taken from understanding the problem to designing, implementing, testing, and documenting the system.

For the **Personal Contact Directory with Search and Update Features** History a structured development approach was followed to ensure that the system meets the requirements and functions efficiently.

The development process includes the following stages:

• Problem Analysis
• System Design
• Program Development
• Testing and Validation
• Result and Documentation

The **Bank Account Management System with Deposit, Withdrawal and Transaction History** follows a structured development approach.

4.1 Problem Analysis

Understanding the need for efficient contact storage and retrieval led to identifying key operations like add, search, update, and delete.

4.2 System Design

The system is designed using structured programming principles. Data structures like arrays, lists, or dictionaries are used to store contacts.

4.3 Program Development

The program is developed using a programming language such as C, C++, Java, or Python. Functions are implemented for each module.

4.4 Testing and Validation

- **Unit testing for each module**
- **Testing with valid and invalid inputs**
- **Ensuring correct search and update functionality**

4.5 Result and Documentation

The system successfully performs all required operations and maintains contact records efficiently.

4.6 Algorithm

Algorithm for Contact Directory:

1. **Start**
2. **Display menu options**
3. **Read user choice**
4. **If choice = Add → Input details and store**
5. **If choice = Search → Input name/number and display result**
6. **If choice = Update → Modify selected contact**
7. **If choice = Delete → Remove contact**
8. **If choice = Display → Show all contacts**
9. **Repeat until exit**
10. **Stop**

4.7 Flowchart

(You can draw this in your record)

Flow:
Start → Menu → Choice →
→ Add / Search / Update / Delete / Display → Back to Menu → End

5.SOURCE CODE

```
import java.util.*;


class Contact {

    String name;

    String phone;

    String email;


    Contact(String name, String phone, String email) {

        this.name = name;

        this.phone = phone;
```

```
        this.email = email;
    }

    public String toString() {
        return "Name: " + name + ", Phone: " + phone + ", Email: " +
email;
    }
}

public class ContactDirectory {
    static ArrayList<Contact> contacts = new ArrayList<>();

    public static void addContact(String name, String phone, String
email) {
        contacts.add(new Contact(name, phone, email));
        System.out.println("Contact added successfully!");
    }

    public static void viewContacts() {
        if (contacts.isEmpty()) {
            System.out.println("No contacts available.");
            return;
        }
        for (Contact c : contacts) {
            System.out.println(c);
        }
    }

    public static void searchContact(String name) {
        boolean found = false;
        for (Contact c : contacts) {
            if (c.name.equalsIgnoreCase(name)) {
                System.out.println("Contact Found: " + c);
```

```
            found = true;
        }
    }
    if (!found) {
        System.out.println("Contact not found.");
    }
}

public static void updateContact(String name, Scanner sc) {
    for (Contact c : contacts) {
        if (c.name.equalsIgnoreCase(name)) {
            System.out.print("Enter new phone: ");
            c.phone = sc.nextLine();
            System.out.print("Enter new email: ");
            c.email = sc.nextLine();
            System.out.println("Contact updated successfully!");
            return;
        }
    }
    System.out.println("Contact not found.");
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int choice;

    do {
        System.out.println("\n--- Contact Directory ---");
        System.out.println("1. Add Contact");
        System.out.println("2. View Contacts");
        System.out.println("3. Search Contact");
        System.out.println("4. Update Contact");
```

```
System.out.println("5. Exit");
System.out.print("Enter your choice: ");

choice = Integer.parseInt(sc.nextLine());

switch (choice) {
    case 1:
        System.out.print("Enter name: ");
        String name = sc.nextLine();
        System.out.print("Enter phone: ");
        String phone = sc.nextLine();
        System.out.print("Enter email: ");
        String email = sc.nextLine();
        addContact(name, phone, email);
        break;

    case 2:
        viewContacts();
        break;

    case 3:
        System.out.print("Enter name to search: ");
        searchContact(sc.nextLine());
        break;

    case 4:
        System.out.print("Enter name to update: ");
        updateContact(sc.nextLine(), sc);
        break;

    case 5:
        System.out.println("Exiting...");
```

```
                    break;


                default:
                    System.out.println("Invalid choice!");
            }
        } while (choice != 5);


        sc.close();
    }
}
```

6.INPUT AND OUTPUT SCREENS

```
1.Add  2.Search  3.Update  4.Delete  5.Display  6.Exit
Enter choice: 1
Enter name: John
Enter phone: 9876543210
Contact added successfully!

Enter choice: 5
Contact List:
John : 9876543210

Enter choice: 2
Enter name to search: John
Phone: 9876543210

Enter choice: 3
Enter name to update: John
Enter new phone: 9998887770
Contact updated successfully!

Enter choice: 4
Enter name to delete: John
Contact deleted successfully!

Enter choice: 6
Exiting program...
```

7.DISCUSSION & CONCLUSION

7.1 Discussion

The system demonstrates how basic programming concepts can be applied to solve real-world problems. It provides an efficient way to manage contacts digitally.

7.2 Conclusion

The Personal Contact Directory successfully meets its objectives by enabling users to store, search, and update contact information easily. It can be further enhanced with advanced features like GUI, database integration, and cloud storage.

8.REFERENCES

The following resources were used during the development of this project:

Books

• Herbert Schildt – *Java: The Complete Reference*

Websites

• https://www.oracle.com/java
• https://www.geeksforgeeks.org
• https://www.w3schools.com
• https://www.javatpoint.com